ChatGPT

docker-compose.yml

```
version: '3'
services:
  db:
    image: pgvector/pgvector:latest
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: events
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./backend/init.sql:/docker-entrypoint-initdb.d/init.sql

  backend:
    build: ./backend
    depends_on:
      - db
    environment:
      DB_HOST: db
      DB_NAME: events
      DB_USER: postgres
      DB_PASSWORD: password
      LLM_HOST: llm
      LLM_PORT: "11434"
      LLM_MODEL: qwen:latest
      XDG_RUNTIME_DIR: /run/user/1000
      WAYLAND_DISPLAY: wayland-0
      DBUS_SESSION_BUS_ADDRESS: unix:path=/run/user/1000/bus
      GNOME_ACCESSIBILITY: "1"
    volumes:
      - /run/user/1000:/run/user/1000

  llm:
    image: ollama/ollama:latest
    ports:
      - "11434:11434"
    volumes:
      - ./models:/root/.ollama
    command: ["ollama", "serve"]
```

```
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend

volumes:
  pgdata:
```

Backend

Dockerfile

```
FROM python:3.11-slim

RUN apt-get update && apt-get install -y \
    python3-pyatspi \
    tesseract-ocr \
    wl-clipboard \
    libglib2.0-0 \
    dbus-x11 \
    && rm -rf /var/lib/apt/lists/*

RUN pip install --no-cache-dir fastapi uvicorn[standard] pillow pytesseract
requests psycopg2-binary sentence-transformers

RUN python -c "from sentence_transformers import SentenceTransformer;
SentenceTransformer('all-MiniLM-L6-v2')"

WORKDIR /app
COPY main.py .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

init.sql

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE events (
    id SERIAL PRIMARY KEY,
    timestamp TIMESTAMP NOT NULL,
```

```
    type VARCHAR(10) NOT NULL,
    content TEXT,
    image_path TEXT,
    source_app VARCHAR(100),
    source_title TEXT,
    summary TEXT,
    tags TEXT
);

CREATE TABLE event_embeddings (
    event_id INT PRIMARY KEY REFERENCES events(id) ON DELETE CASCADE,
    embedding vector(384)
);

CREATE TABLE event_metadata (
    event_id INT REFERENCES events(id) ON DELETE CASCADE,
    key TEXT,
    value TEXT
);
```

main.py

```
import os
import threading
import subprocess
import json
import datetime
import time
import sys
import hashlib
import io
import uuid

import pyatspi
import pytesseract
from PIL import Image
import requests
import psycopg2
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from starlette.responses import FileResponse
from sentence_transformers import SentenceTransformer

# Global state
current_app = None
current_title = None
```

```
last_text = None
last_image_hash = None
db_lock = threading.Lock()

# Load embedding model
embed_model = SentenceTransformer('all-MiniLM-L6-v2')

# Config from environment
db_host = os.getenv("DB_HOST", "db")
db_name = os.getenv("DB_NAME", "events")
db_user = os.getenv("DB_USER", "postgres")
db_password = os.getenv("DB_PASSWORD", "password")
llm_host = os.getenv("LLM_HOST", "llm")
llm_port = os.getenv("LLM_PORT", "11434")
llm_model = os.getenv("LLM_MODEL", "qwen:latest")

# Initialize FastAPI
app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_credentials=True,
allow_methods=["*"], allow_headers=["*"])

# Database connection
conn = None

def call_llm_and_parse(text):
    prompt = f"Provide a brief summary and comma-separated tags for the
following content:\\n\"\"\"\\n{text}\\n\"\"\"\\nRespond in JSON with keys
'summary' and 'tags'."
    payload = {"model": llm_model, "prompt": prompt, "options": {"format":
"json"}, "stream": False}
    try:
        res = requests.post(f"http://{llm_host}:{llm_port}/api/generate",
json=payload, timeout=30)
        res.raise_for_status()
        # Try to parse JSON response
        try:
            data = res.json()
        except ValueError:
            data = None
        if data and isinstance(data, dict) and "summary" in data and "tags" in
data:
            summary = data.get("summary", "")
            tags = data.get("tags", "")
        else:
            # Fallback to parse text output
            text_out = res.text.strip()
            try:
                parsed = json.loads(text_out)
```

```
                summary = parsed.get("summary", "")
                tags = parsed.get("tags", "")
            except Exception:
                summary = text_out
                tags = ""
        return summary, tags
    except Exception as e:
        print("LLM call failed:", e)
        return "", ""

def handle_event(event_type, content, image_path=None):
    global conn
    src_app = current_app
    src_title = current_title
    timestamp = datetime.datetime.now()
    summary = ""
    tags = ""
    # Determine if we should call LLM
    if event_type == "image":
        if content and content.strip() != "":
            summary, tags = call_llm_and_parse(content)
        else:
            summary = "Image with no text"
            tags = "image"
    else:
        summary, tags = call_llm_and_parse(content)
    # Generate embedding
    embedding = None
    if content and content.strip() != "":
        vec = embed_model.encode(content)
        try:
            vec = vec.tolist()
        except AttributeError:
            vec = list(vec)
        embedding = vec
    # Insert into database
    event_id = None
    with db_lock:
        try:
            cur = conn.cursor()
            cur.execute("INSERT INTO events (timestamp, type, content,
image_path, source_app, source_title, summary, tags) VALUES (%s,%s,%s,%s,%s,%s,
%s,%s) RETURNING id",
                        (timestamp, event_type, content, image_path, src_app,
src_title, summary, tags))
            event_id = cur.fetchone()[0]
            if embedding is not None:
                vec_text = "[" + ",".join(str(x) for x in embedding) + "]"
```

```
cur.execute(f"INSERT INTO event_embeddings (event_id, embedding) VALUES (%s,
'{vec_text}')", (event_id,))
            conn.commit()
        except Exception as e:
            conn.rollback()
            print("DB error:", e)
    # Log the event as JSON
    event_data = {
        "id": event_id,
        "timestamp": timestamp.isoformat(),
        "type": event_type,
        "content": content,
        "image_path": image_path,
        "source_app": src_app,
        "source_title": src_title,
        "summary": summary,
        "tags": tags
    }
    print(json.dumps(event_data))

def watch_clipboard_text():
    global last_text
    proc = subprocess.Popen(["wl-paste", "--watch", "--type", "text", "cat"],
stdout=subprocess.PIPE)
    while True:
        line = proc.stdout.readline()
        if not line:
            if proc.poll() is not None:
                break
            else:
                continue
        try:
            text = line.decode("utf-8", errors="ignore").rstrip()
        except Exception:
            text = ""
        if text == "" or text == last_text:
            continue
        last_text = text
        handle_event("text", text)

def watch_clipboard_image():
    global last_image_hash
    proc = subprocess.Popen(["wl-paste", "--watch", "--type", "image", "cat"],
stdout=subprocess.PIPE)
    while True:
        data = proc.stdout.read()
        if not data:
```

```
            if proc.poll() is not None:
                break
            else:
                continue
        img_hash = hashlib.md5(data).hexdigest()
        if img_hash == last_image_hash:
            continue
        last_image_hash = img_hash
        try:
            img = Image.open(io.BytesIO(data))
        except Exception as e:
            print("Error reading image:", e)
            continue
        os.makedirs("/tmp/activity_images", exist_ok=True)
        img_filename = f"{uuid.uuid4()}.png"
        img_path = os.path.join("/tmp/activity_images", img_filename)
        try:
            img.save(img_path, format="PNG")
        except Exception as e:
            print("Error saving image:", e)
        ocr_text = ""
        try:
            ocr_text = pytesseract.image_to_string(img).strip()
        except Exception as e:
            print("OCR failed:", e)
        handle_event("image", ocr_text, image_path=img_filename)

def focus_event(event):
    try:
        if event.type.startswith("object:state-changed:focused") or
event.type.startswith("object:state-changed:active"):
            if event.detail1 == 1:
                app_name = ""
                title = ""
                try:
                    app_name = str(event.host_application.name)
                except:
                    app_name = ""
                if app_name and app_name.lower() != "gnome-shell":
                    try:
                        title = str(event.source.name)
                    except:
                        title = ""
                    global current_app, current_title
                    current_app = app_name
                    current_title = title if title else app_name
    except Exception:
        pass
```

```
def start_focus_monitor():
    try:
        pyatspi.Registry.registerEventListener(focus_event, "object:state-
changed:focused")
        pyatspi.Registry.registerEventListener(focus_event, "object:state-
changed:active")
        pyatspi.Registry.start()
    except Exception as e:
        print("Focus monitor error:", e)

@app.on_event("startup")
def on_startup():
    global conn
    # Connect to PostgreSQL (retry until available)
    for _ in range(10):
        try:
            conn = psycopg2.connect(host=db_host, dbname=db_name, user=db_user,
password=db_password)
            break
        except Exception as e:
            print("DB connection failed, retrying...", e)
            time.sleep(2)
    if conn is None:
        print("Could not connect to database, exiting.")
        sys.exit(1)
    # Start background threads
    threading.Thread(target=watch_clipboard_text, daemon=True).start()
    threading.Thread(target=watch_clipboard_image, daemon=True).start()
    threading.Thread(target=start_focus_monitor, daemon=True).start()

@app.get("/api/events")
def get_events():
    with db_lock:
        cur = conn.cursor()
        cur.execute("SELECT id, timestamp, type, content, image_path,
source_app, source_title, summary, tags FROM events ORDER BY timestamp")
        rows = cur.fetchall()
    return [
        {
            "id": row[0],
            "timestamp": row[1].isoformat(),
            "type": row[2],
            "content": row[3],
            "image_path": row[4],
            "source_app": row[5],
            "source_title": row[6],
            "summary": row[7],
```

```
            "tags": row[8]
        }
        for row in rows
    ]

@app.get("/api/image/{filename}")
def get_image(filename: str):
    file_path = os.path.join("/tmp/activity_images", filename)
    if not os.path.exists(file_path):
        raise HTTPException(status_code=404, detail="Image not found")
    return FileResponse(file_path, media_type="image/png")
```

Frontend

Dockerfile

```
FROM node:18 AS build
WORKDIR /app
COPY package.json ./
RUN npm install
COPY public ./public
COPY src ./src
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
```

package.json

```
{
  "name": "event-timeline",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
```

```
  }
}
```

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Event Timeline</title>
</head>
<body>
  <div id="root"></div>
</body>
</html>
```

src/index.js

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

const container = document.getElementById('root');
const root = createRoot(container);
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

src/App.js

```
import React, { useState, useEffect } from 'react';

function App() {
  const [events, setEvents] = useState([]);

  useEffect(() => {
    fetch('/api/events')
      .then(res => res.json())
      .then(data => setEvents(data))
```

```
      .catch(err => console.error('Error fetching events:', err));
  }, []);

  return (
    <div className="App">
      <h1>Event Timeline</h1>
      <ul>
        {events.map(event => (
          <li key={event.id}>
            <div>
              <strong>{new Date(event.timestamp).toLocaleString()}</strong> –
<em>{event.source_app}</em> – {event.source_title}
            </div>
            {event.type === 'text' ? (
              <div>"{event.content}"</div>
            ) : (
              <div>
                <img src={`/api/image/${event.image_path}`} alt="clipboard"
style={{ maxWidth: '200px', maxHeight: '150px' }} />
                {event.content ? <div>OCR Text: "{event.content}"</div> :
<div><em>No text detected</em></div>}
              </div>
            )}
            <div>Summary: {event.summary}</div>
            <div>Tags: {event.tags}</div>
            <hr />
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

src/index.css

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
}
ul {
  list-style-type: none;
  padding: 0;
}
li {
```

```
  margin-bottom: 20px;
}
```